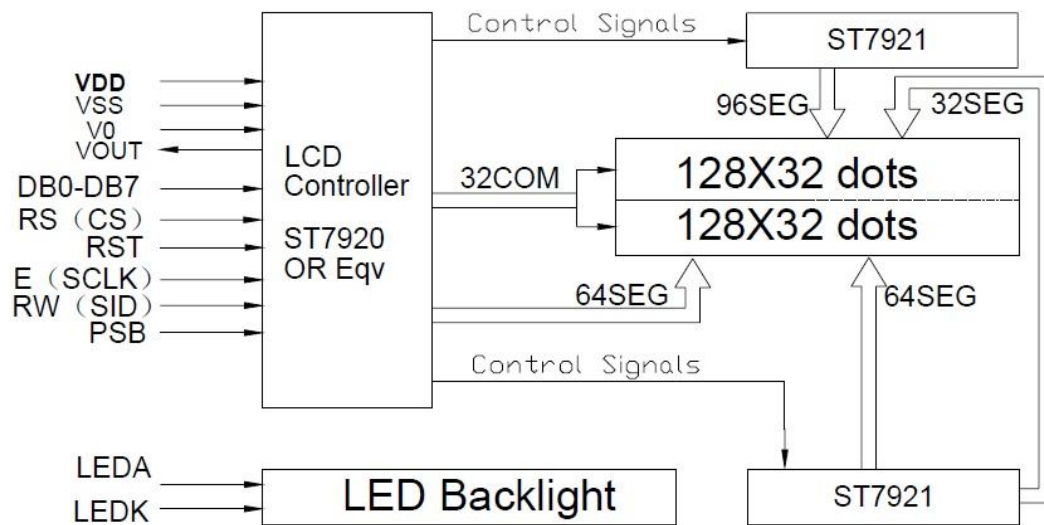


LCD12864 液晶显示实验-汉字及自定义显示(并口)

LCD12864 带字库，型号：CM12864-12.其相关数据手册我们已经提供在资料中为“ST7920 系列中文图形液晶模块使用说明书”，里面有详细的介绍。这里就不在多描述。其原理简图：（我们只需关心接口部分）

128×64 系列原理简图



接口定义：[这里注意 V0](#)，我一开始就栽倒这，网上搜到的资料中，这 PIN 可以悬空或接个滑动电阻，程序检查好多遍，没有发现问题，就是不显示，接到 3.3V 上也不行，后来把 V0 直接接到 5V 上，就可以正常显示。在这提醒大家数据手册仅供参考，具体还是以自己的实物为主。避免走没必要的弯路。

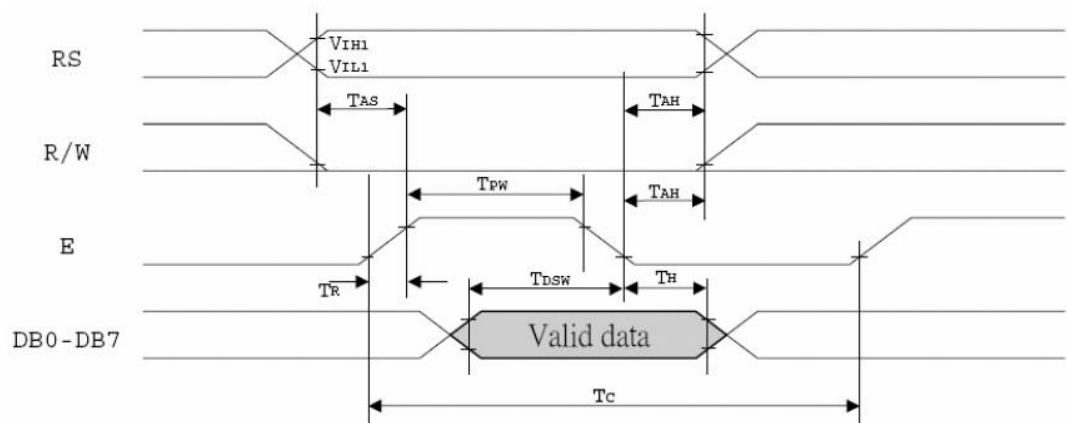
管脚号	管脚名称	电平	管脚功能描述
1	VSS	0V	电源地
2	VCC	5V	电源正
3	V0	-	对比度（亮度）调整. (直接接到 5V 上最靠谱)

4	RS(CS)	H/L	RS= “H” ,表示 DB7——DB0 为显示数据 RS= “L” ,表示 DB7——DB0 为显示指令数据
5	R/W(SID)	H/L	R/W= “H” ,E= “H” ,数据被读到 DB7——DB0 R/W= “L” ,E= “H→L” , DB7——DB0 的数据被写到 IR 或 DR
6	E(SCLK)	H/L	使能信号
7	DB0	H/L	三态数据线
8	DB1	H/L	三态数据线
9	DB2	H/L	三态数据线
10	DB3	H/L	三态数据线
11	DB4	H/L	三态数据线
12	DB5	H/L	三态数据线

13	DB6	H/L	三态数据线
14	DB7	H/L	三态数据线
15	PSB	H/L	H: 8 位或 4 位并口方式, L: 串口方式
16	NC	-	空脚
17	/RESET	H/L	复位端, 低电平有效 (悬空或接到 5V 都可以)
18	VOUT	-	LCD 驱动电压输出端(悬空)
19	A	VDD	背光源正端 (+5V)
20	K	VSS	背光源负端(0V)

ST7920 数据手册中,提到每次在写命令之前都要判断 LCD 是否处于空闲状态,是为 0, 否则为 1, 本次试验全过程只写不读, 为了避开 LCD 忙的状态, 可以采取隔断时间写一次命令或数据。在指令表中, 可以看到每个命令执行的时间, 这里注意, 除清除显示和地址归位, 网上有不同的说法, 有的说是 1.6ms, 有的 4.6ms 外, 其他命令执行时间都是 72us, 不管怎么样, 我们还是取最大值更为靠谱, 4.6ms 取个整数, 好听点的 6ms 吧。当然可以取比这个更大 (LCD 在显示数据时, 是一个接一个的显示出来, 就可以察觉到, 取的越大, 显示速度就越慢), 看自己的决定了。利用 FPGA 分频, 产生一个周期 6ms 的时钟, 给 LCD 的 E 管脚, 进行控制写数据或命令。在 8 为并口写操作时序图中, DB0~DB7 是在 E 为高时, 写入数据。

8 位并口写操作时序图



知道了何时给 LCD 写数据或命令，还得知如何把自己想要显示的东西显示在 LCD 屏幕上，这点也不难：

第一得知道中文字字的编码，可以查国标 GB2312，来调用 CGROM 字库，比如“文:CEC4、少: C9D9、清: C7E5 ”。或可以下载“汉字十六进制转换工具”软件直接生成。第二得知道显示文字的地址该如何设置，就是一个文字我可以摆在第一行的开头或结尾或其他行。如下图示。

80H↕	81H↕	82H↕	83H↕	84H↕	85H↕	86H↕	87H↕
90H↕	91H↕	92H↕	93H↕	94H↕	95H↕	96H↕	97H↕
88H↕	89H↕	8AH↕	8BH↕	8CH↕	8DH↕	8EH↕	8FH↕
98H↕	99H↕	9AH↕	9BH↕	9CH↕	9DH↕	9EH↕	9FH↕

第三若自带的库中没有你想要的东西时，可以自己定义，比如代码中的星星月亮组合图标，库中没有，那么我们可以自己定义（组合图标的编码可以用字模软件进行提取）。自定义字符具体操作，先设定 CGRAM 存储地址，其次把该组合图标的编码数据放置到该地址中，在设置该图标在 LCD 屏幕显示的位置，最后连续写两次数据（**注意写的是数据不是命令，我也被这东东栽倒一次**），先写高 8bit，后写低 8bit，就可以了。此时知道写这数据到底是啥数据。那么就理解 0x0000,0x0002,0x0004,0x0006 这四种编码。至于这四种编码有什么区别，我也不是很明白，在程序中，0x0000 和 0x0006 没发现有区别，显示正常；0x0002，在星星的左上角有个小点；0x0004 显示乱码。

第四半宽字型，数据手册中写的是“半宽字型 ROM(HCGROM)，总共提供 126 个西文字型（16×8 点阵）”，应该就是英文字符。程序中的“LCD12864”。

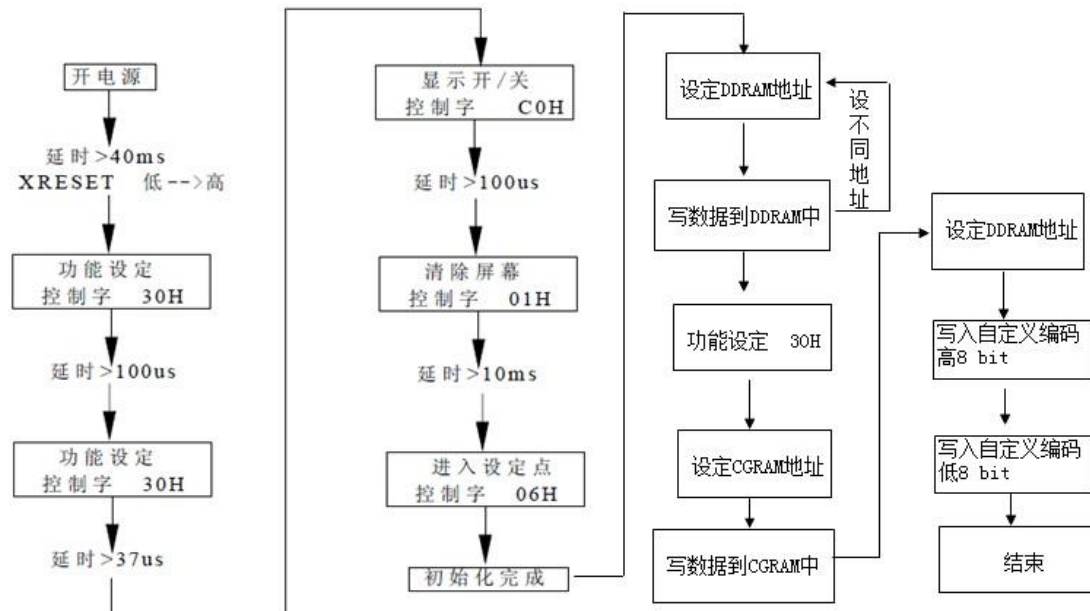
显示 RAM(DDRAM)

显示 RAM 提供 64×2 字节的空间, 最多可以控制 4 行 16 字的中文字型显示。当写入显示资料 RAM 时, 可以分别显示 CGROM, HCGROM 及 CGRAM 的字型。

三种字型的选择:

- 1) 显示半宽字型 将一个字节的编码写入 DDRAM 中, 范围是 $02 \sim 7FH$
- 2) 显示 CGRAM 字型 将 2 个字节的编码写入 DDRAM 中, 共有 $0000H$, $0002H$, $0004H$ 及 $0006H$ 四种编码
- 3) 显示中文字型 将 2 字节的编码写入 DDRAM 中, 先写高 8 位, 后写低 8 位范围是 $A140H \sim D75FH$ (BIG5), $A1A0H \sim F7FFH$ (GB)

在写代码之前理清思路, 程序的流程该如何跑, 写起代码来也比较顺手。如下图。前面设置好频率后, 每步中的延时就不用考虑了。



代码: (使用并口, 只有写)

LCD12864.v

```
module LCD_12864(
    //input
    sys_clk,
    rst_n,
    //output
    lcd_rs,
    lcd_rw,
    lcd_en,
    lcd_data,
```

```

        lcd_psb
    );
input sys_clk;// 50MHZ
input rst_n;

output lcd_rs;//H:data    L:command
output lcd_rw;//H:read module    L:write module
output lcd_en;//H active
output [7:0] lcd_data;
output lcd_psb;//H:parallel    module    L:SPI module

/*****/
parameter T3MS = 18'd149_999;
parameter    IDLE            = 4'd0,
            INIT_FUN_SET1 = 4'd1,
            INIT_FUN_SET2 = 4'd2,
            INIT_DISPLAY  = 4'd3,
            INIT_CLEAR    = 4'd4,
            INIT_DOT_SET  = 4'd5,
            SET_DDRAM     = 4'd6,
            WRITE_DATA1   = 4'd7,
            INIT_FUN_SET3 = 4'd8,
            SET_CGRAM     = 4'd9,
            WRITE_DATA2   = 4'd10,
            SET_DDRAM2    = 4'd11,
            SET_CUSTOM_L  = 4'd12,
            SET_CUSTOM_H  = 4'd13,
            STOP          = 4'd14;
/*****/
//产生周期为 6MS 的 lcd_clk 给 LCD
reg [17:0] cnt;
reg lcd_clk;
always @(posedge sys_clk or negedge rst_n)
if(!rst_n) begin

```

```

        cnt <= 18'd0;
        lcd_clk <= 1'b0;
    end
    else if(cnt == T3MS)begin
        cnt <= 18'd0;
        lcd_clk <= ~lcd_clk;
    end
    else
        cnt <= cnt + 1'b1;
    /*****/
    reg lcd_rs;
    always @(posedge lcd_clk or negedge rst_n)
    if(!rst_n)
        lcd_rs <= 1'b0;
    else if((state == WRITE_DATA1) || (state == WRITE_DATA2) || (state ==
    SET_CUSTOM_H) || (state == SET_CUSTOM_L))
        lcd_rs <= 1'b1;        //写数据模式
    else
        lcd_rs <= 1'b0;        //写命令模式
    /*****/
    reg [3:0] state;
    reg [7:0] lcd_data;
    reg [6:0] num;
    reg en;
    always @(posedge lcd_clk or negedge rst_n)
    if(!rst_n) begin
        state <= IDLE;
        lcd_data <= 8'hzz;
        en <= 1'b1;
        num <= 6'd0;
    end
    else
        case(state)
            IDLE:

```

```

begin
    state <= INIT_FUN_SET1;
    lcd_data <= 8'hzz;
    en <= 1'b1;
end

INIT_FUN_SET1:
begin
    lcd_data <= 8'h30;    //功能设定
    state <= INIT_FUN_SET2;
end

INIT_FUN_SET2:
begin
    lcd_data <= 8'h30;    //功能设定
    state <= INIT_DISPLAY;
end

INIT_DISPLAY:
begin
    lcd_data <= 8'h0c;    //显示设定
    state <= INIT_CLEAR;
end

INIT_CLEAR:
begin
    lcd_data <= 8'h01;    //清屏
    state <= INIT_DOT_SET;
end

INIT_DOT_SET:
begin
    lcd_data <= 8'h06;    //进入点设定
    state <= SET_DDRAM;
end

SET_DDRAM:
begin

```

```

if(num == 7'd0)
    lcd_data <= 8'h80;//1 line
else if(num == 7'd16)
    lcd_data <= 8'h91;//2 line
else if(num == 7'd28)
    lcd_data <= 8'h88;//3 line
else if(num == 7'd42)
    lcd_data <= 8'h98;//4 line
state <= WRITE_DATA1;
end
WRITE_DATA1:
begin
    if(num <= 7'd15) begin
        num <= num + 1'b1;
        lcd_data <= dis_data;
        if(num == 7'd15)
            state <= SET_DDRAM;
        else
            state <= WRITE_DATA1;
    end
    else if( num > 7'd15 && num <= 7'd27) begin
        num <= num + 1'b1;
        lcd_data <= dis_data;
        if(num == 7'd27)
            state <= SET_DDRAM;
        else
            state <= WRITE_DATA1;
    end
    else if(num > 7'd27 && num <= 7'd41) begin
        num <= num + 1'b1;
        lcd_data <= dis_data;
        if(num == 7'd41)
            state <= SET_DDRAM;
        else

```

```

        state <= WRITE_DATA1;
    end
    else if(num > 7'd41 && num <= 7'd57) begin
        num <= num + 1'b1;
        lcd_data <= dis_data;
        if(num == 7'd57)
            state <= INIT_FUN_SET3;
        else
            state <= WRITE_DATA1;
        end
    end
end
INIT_FUN_SET3:
begin
    lcd_data <= 8'h30;//功能设定
    state <= SET_CGRAM;
end
SET_CGRAM:
begin
    lcd_data <= 8'h40;//设定 CGRAM 字符的位置
    state <= WRITE_DATA2;
end
WRITE_DATA2:
begin
    if(num >= 7'd42 && num <= 7'd57) begin
        num <= num + 1'b1;
        lcd_data <= dis_data;
        state <= STOP;
    end
    else begin
        num <= 7'd0;
        state <= STOP;
    end
end
end
// SET_DDRAM2:

```

```

    // begin
    //     lcd_data <= 8'h9e;    //4line
    //     state <= SET_CUSTOM_H;//设置自定义显示字符编码
    // end
//
// SET_CUSTOM_H:
// begin
//     lcd_data <= 8'h00;//高 8bit
//     state <= SET_CUSTOM_L;
// end
//
// SET_CUSTOM_L:
// begin
//     lcd_data <= 8'h00;//低 8bit 00:正常 02: 星星左上角多了一小点 04: 显示
//     state <= STOP;
// end

    STOP:
    begin
        en <= 1'b0;//显示完了, lcd_e 就一直拉为低
        state <= STOP;
    end

    default: state <= IDLE;
endcase
/*****/
reg [7:0] dis_data;
always @(posedge sys_clk or negedge rst_n)
if(!rst_n)
    dis_data <= 8'hzz;
else
    case(num)
        //1 Line 80 位置显示 杭州康芯电子公司

```

7'd0 : dis_data <= 8'hBA;//显示第一行 // 杭
7'd1 : dis_data <= 8'hBC;

7'd2 : dis_data <= 8'hD6; //州
7'd3 : dis_data <= 8'hDD;

7'd4 : dis_data <= 8'hBF;//康
7'd5 : dis_data <= 8'hB5;

7'd6 : dis_data <= 8'hD0;// 芯
7'd7 : dis_data <= 8'hBE;

7'd8 : dis_data <= 8'hB5;//电
7'd9 : dis_data <= 8'hE7;

7'd10 : dis_data <= 8'hD7; //子
7'd11 : dis_data <= 8'hD3;
7'd12 : dis_data <= 8'hB9;// 公
7'd13 : dis_data <= 8'hAB;
7'd14 : dis_data <= 8'hCB;//司
7'd15 : dis_data <= 8'hBE;

//2Line 91 位置显示 热情为您服务

7'd16 : dis_data <= 8'hC8;//热
7'd17 : dis_data <= 8'hC8;
7'd18 : dis_data <= 8'hC7;//情
7'd19 : dis_data <= 8'hE9;
7'd20 : dis_data <= 8'hCE;//为
7'd21 : dis_data <= 8'hAA;
7'd22 : dis_data <= 8'hC4;//您
7'd23 : dis_data <= 8'hFA;

7'd24 : dis_data <= 8'hB7;//服
7'd25 : dis_data <= 8'hFE;
7'd26 : dis_data <= 8'hCE;//务

7'd27 : dis_data <= 8'hF1;

//3Line 88 位置显示 www.kx-soc.com

7'd28 : dis_data <= "w";

7'd29 : dis_data <= "w";

7'd30 : dis_data <= "w";

7'd31 : dis_data <= ".";

7'd32 : dis_data <= "k";

7'd33 : dis_data <= "x";

7'd34 : dis_data <= "-";

7'd35 : dis_data <= "s";

7'd36 : dis_data <= "o";

7'd37 : dis_data <= "c";

7'd38 : dis_data <= ".";

7'd39 : dis_data <= "c";

7'd40 : dis_data <= "o";

7'd41 : dis_data <= "m";

//4Line 98 位置显示电话号!

7'd42 : dis_data <= "T";

7'd43 : dis_data <= "E";

7'd44 : dis_data <= "L";

7'd45 : dis_data <= "0";

7'd46 : dis_data <= "5";

7'd47 : dis_data <= "7";

7'd48 : dis_data <= "1";

7'd49 : dis_data <= "-";

7'd50 : dis_data <= "8";

7'd51 : dis_data <= "8";

7'd52 : dis_data <= "9";

7'd53 : dis_data <= "1";

7'd54 : dis_data <= "4";

7'd55 : dis_data <= "5";

7'd56 : dis_data <= "8";

7'd57 : dis_data <= "7";

default: dis_data <= 8'h00;

```
//4Line 99 位置显示 谢谢!  
/*****  
assign lcd_rw = 1'b0;//只有写模式  
assign lcd_psb = 1'b1;//并口模式  
assign lcd_en = en ? lcd_clk : 1'b0;  
*****/  
endmodule
```

硬件验证:

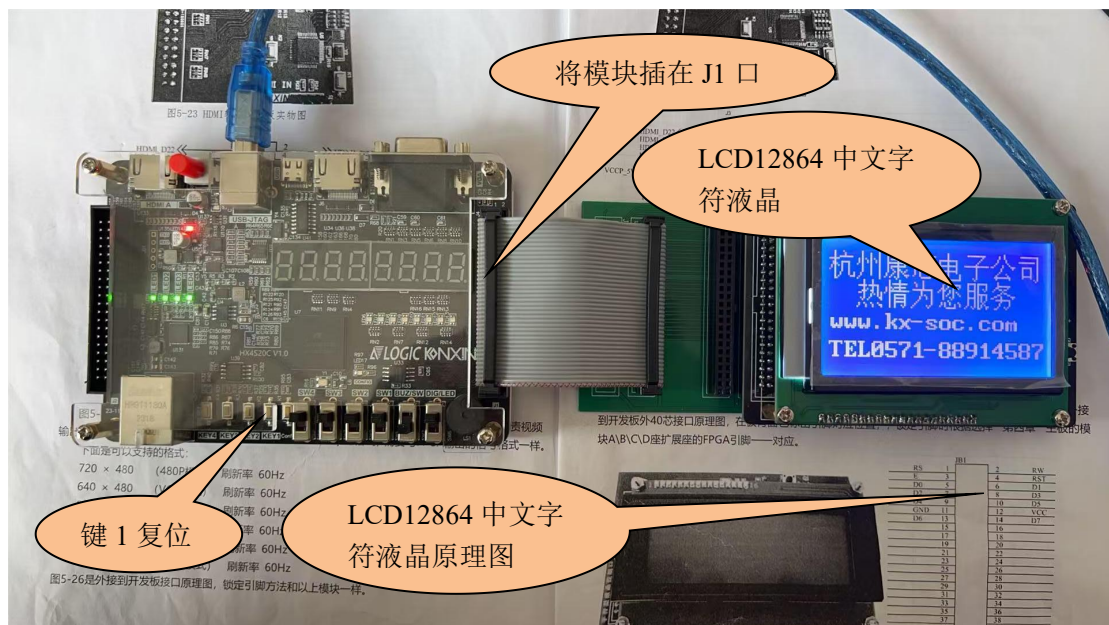


图5-25 HDMI输出扩展板

